

Notes on Information Theory

Riccardo Lo Iacono

May 15, 2026

Contents.

1	Introduction	1
1.1	Information and Shannon's theory	1
1.2	Encoding a source	2
1.3	The Kraft-McMillan inequality	4
1.4	Average codeword length	5
2	Huffman encoding	7
2.1	Canonical Huffman encoding	8
2.2	Adaptive Huffman encoding	10
3	Arithmetic encoding	11
3.1	Adaptive arithmetic encoding	12
4	Universal encoding of integers	14
4.1	Elias codes	14
4.2	Fibonacci codes	16
5	Burrows-Wheeler transform	18
5.1	Analysis of the BWT	19
5.2	The DC3 algorithm	22
	References	24
	Acronyms	25

– 1 – Introduction.

At the core of modern computer science, among others, two fields have become fundamental: *information theory* and *data compression*. The former deals on how informations can be transmitted through a channel; the latter with methods to maximize the throughput of the channel. More correctly, data compression studies how to efficiently encode data; that is, how to store informations efficiently.

In what follows, we discuss topics of both fields in parallel; we choosed this approach as the two fields are closely related.

– 1.1 – Information and Shannon’s theory.

Above we’ve used the term information assuming its meaning as given; it should be clear that the notion of information in the spoken language has little to do with the CS counterpart.

Formally, the notion of information is that of a measure of the reduction of the uncertainty of the outcomes of an event. That is, a measure of how much knowledge is gained observing the event. Therefore, it makes sense that the information carried by an event is inversely proportional to its probability.

Historically, the first person attempting to define information was R.V.L. Hartley; see [7] for more details. But the major contribution is due to Claude Shannon, who in [12], building on Hartley’s ideas, provides a rigorous definition of information and a way to compute it.

In his paper, Shannon shows that any communication system can be reduced to five parts: a *sender*, a *transmitter*, a *communication channel*, a *receiver* and a *destination*. A schematic representation of such system is shown in Figure 1.

With his paper, Shannon introduces introduces two important hypotesis:

- Transmitting a symbol requires a non-zero amount of time;
- The communication system is subject to noise.

Remark: In his paper Shannon actually develops two theories: one with noise and one without. We focus on the latter.

The question now is: what properties should a measure of information satisfy? According to Shannon, such a function should have a dependence on probability, should be monotonic, continous and additive for indenpendent events.

More often than not, we are interestend in computing the information a source; that is, an entity over some alphabet Σ , that at any given time produces a symbol $\sigma \in \Sigma$ with probability $\Pr(\sigma)$. For instance, a Markov chain

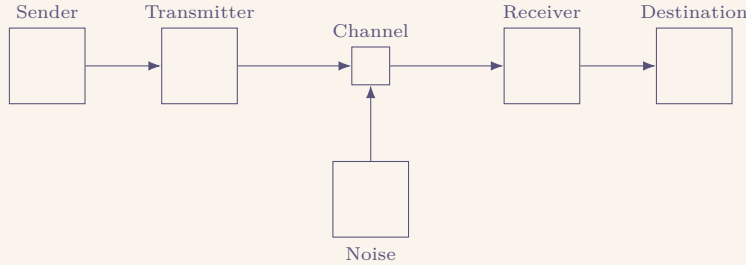


Figure 1: Schematic representation of Shannon’s communication system.

is an example of source of information. We shall say that there exist two types of sources: with memory and memoryless. In what follows, we consider memoryless sources.

But how do we actually measure information? According to Shannon, the only valid measure is

$$H(S) = K \sum_{i=1}^n p_i \log_b p_i \quad (1)$$

where n is the total number of symbols produced by S , p_i the probability associated with the i -th symbol produced, and K a positive constant usually set to 1. Unless stated otherwise, the base b of the logarithm is always 2. We usually refer to Equation (1) as *entropy of a source S* .

Although we could prove that Equation (1) is the only valid measure, it is beyond our scope; the interested reader is referred to [12][Appendix 2].

– 1.2 – Encoding a source.

A problem arises with sources: the alphabet these use are, in general, not suitable for transmission. Is for this reason that, as shown in Figure 1, we make use of transmitters and receivers. Although we describe only the encoding process of a source, the decoding is trivially symmetric.

Let S be a source. A code is defined as a map from the source alphabet to a new alphabet, called input alphabet. In Sections 2 to 4 we consider some particular family of codes, while in the remainder of this section we introduce some classes of code, as well as some theoretical tools useful for studying codes.

– 1.2.1 – Non-singular codes.

In the previous section, we’ve defined a code as a map from the source alphabet to a new one. The question is: are all code equal? It’s easily understood that this is not the case; some code are in fact useless to our needs. For instance, if we define a code that maps two distinct character to the same codeword, the

receiver will have a hard time decoding the text. A code whose codewords are all distinct is called *non-singular*.

– **1.2.2 – Uniquely decodable codes.**

Non-singular codes are not enough. More often than not, it happens that the sequence of bit in analysis may be factored as concatenation of distinct codewords, one again making the decoding process tedious.

Briefly speaking, what we are looking for are codes for which any sequence of codewords corresponds to at most one sequence of source symbol. We call such code Uniquely Decodable (UD) codes.

Hence the questions: how do we construct UD codes? How do we determine if a given code is UD? In the present section we answer to the latter question; the answer to the first question will be discussed later.

In 1953, Sardinas and Patterson proved a result that defines an implicit algorithm to determine if a code is UD. The result, stated below, is based on a simple observation: if we try to construct two factorizations, each word in the first factorizations is either a sub-word in the other factorization or has a prefix that is a suffix of the words in the other factorization.

Theorem (Sardinas-Patterson): *Let $\mathcal{C}$ be a code over the alphabet Σ . Define the sets $S_0, S_1, \dots$ as follow:*

$$S_0 = \mathcal{C}$$

$$S_i = \{w \in \Sigma^* \mid \exists \alpha \in S_0, \exists \beta \in S_{i-1} : \alpha = \beta w \vee \beta = \alpha w\}.$$

Then, necessary and sufficient condition for $\mathcal{C}$ to be UD is that, for all $n > 0$, $S_0 \cap S_n = \emptyset$.

Algorithmically speaking, since the number of possible sets it's finite, the process will eventually stop. The condition in the above theorem, defines three possible halt conditions, namely:

1. $\exists i \geq 0$ s.t. $S_0 \cap S_i \neq \emptyset$, meaning the code is not UD;
2. $\exists i \geq 0$, s.t. $S_i = \emptyset$, making the code UD; and
3. $\exists i, j \geq 0$ s.t. $S_i = S_j$, meaning the code is UD.

In the example following we use colours to show the relationship between α, β and w . Additionally, we align each w to the row of the corresponding α .

Section 1 – Introduction

Example: Consider the code $\mathcal{C} = \{a, c, ad, abb, bad, deb, bbcde\}$, is it UD? Let's apply the Sardinas-Patterson algorithm and check. What we get is the following table.

S_0	S_1	S_2	S_3	S_4	S_5
a					
c			de		
ad	d				
abb	bb				ab
bad					
deb		eb		b	
bbcde		cde			bcde

Since $S_0 \cap S_5 = \{ab\} \neq \emptyset$, we can conclude that $\mathcal{C}$ is not UD.

– 1.2.3 – Prefix codes and the Kraft-McMillan inequality.

One of the most useful properties a code can have is that of being prefix-free; that is, no two codewords are prefix of one another. Thanks to this property, each codeword can be decoded unambiguously, i.e. the word is decoded instantaneously. We call codes with this property *prefix-free*, or simply prefix. We also use the term instantaneous interchangeably.

For this class of code the following results hold.

Theorem: *A code is instantaneous if and only if it's prefix-free.*

Theorem: *All prefix-free codes are UD.*

Proof. Follows as a consequence of Sardinas-Patterson theorem. ■

– 1.3 – The Kraft-McMillan inequality.

We now consider an interesting result about UD codes: the Kraft-McMillan inequality. Formally, the Kraft-McMillan theorem states that a code $\mathcal{C}$ over an alphabet of size d , with of codewords lengths $l_1, \dots, l_n$ is UD, if and only if

$$\sum_{i=1}^n d^{-l_i} \leq 1.$$

The inequality provides therefore a necessary and sufficient condition for a code to be UD.

Proof. We prove the result by construction; that is, we show that if a set of lengths satisfies the inequality, then we can build a tree whose paths from the root to the leaves define codewords of a prefix-free code.

$\Leftarrow$ Easily seen with any UD code.

Section 1 – Introduction

$\Rightarrow$ Let $l_1, \dots, l_n$ be such that

$$\sum_{i=1}^n d^{-l_i} \leq 1.$$

Denote by n_j the number of codewords of length j . Let L be the maximum length. Then

$$\sum_{j=1}^L n_j = n$$

since there are n codewords. Hence, we have

$$\sum_{i=1}^n d^{-l_i} = \sum_{j=1}^L n_j d^{-j} \leq 1.$$

By doing so, we build a d -ary tree which has n_1 leaves at depth 1, n_2 leaves at depth 2, etc. It's easily seen that this produces a prefix-free code. ■

Remark: If the set of lengths is such that the sum is strictly less than one, then some paths in the tree will correspond to no codeword.

– 1.4 – Average codeword length.

We now turn our attention to a more interesting question: fixed a source over some alphabet, how do we design the optimal code for it? The question, apparently hard, has a somewhat obvious answer: take in account the lengths of the codewords. Or better said, their average.

Formally speaking, we take in account the Average Codeword Length (ACL) of the code, defined as the quantity

$$L_S(\mathcal{C}) = \sum_{i=1}^n p_i l_i$$

such that is minimal. We say that a code $\mathcal{C}$ is compact (or optimal) for the source S if, among all prefix codes for S , $\mathcal{C}$ has the minimal ACL.

Remark: Is there a lower bound for the ACL? Yes, and it given by the following result.

Lemma (Shannon): *If $\mathcal{C}$ is a UD code for a source S with probabilities $p_1, \dots, p_n$, and d is the size of the code alphabet, then*

$$L_S(\mathcal{C}) \geq \frac{H(S)}{\log d}$$

Section 1 – Introduction

In addition to the ACL, two additional quantities are significant when designing a code: the efficiency $\vartheta_S(\mathcal{C})$ and the redundancy $\rho_S(\mathcal{C})$. More precisely, we want that the efficiency, defined as

$$\vartheta_S(\mathcal{C}) = \frac{H(S)}{L_S(\mathcal{C})}$$

is maximized; while the redundancy, defined as

$$\rho_S(\mathcal{C}) = 1 - \vartheta_S(\mathcal{C}) = \frac{L_S(\mathcal{C}) - H(S)}{L_S(\mathcal{C})}$$

is minimized.

A question still remains: how do we build optimal codes? Over the years, several attempts to provide a procedure to build optimal codes were done, but until 1952 none achieved the goal. We discuss this procedure in the next section.

– 2 – Huffman encoding.

The first method to build optimal codes was discovered by D. A. Huffman, as solution to a problem posed by his lecturer Robert Fano¹.

Before we discuss such a method, let us observe some properties of optimal codes:

1. $\forall i, j, p_i \geq p_j \implies |c_i| \leq |c_j|$. We could otherwise swap the codeword and obtain a code with a lower ACL.
2. The least probable symbols have the same length. It's easily noted that if not, we could find a code with a lower ACL.
3. The longest codewords differ on the last symbols. It follows directly from the previous property.

The procedure itself is pretty straightforward; it builds a d -ary tree, d being the size of the code alphabet, label each brach with a different symbols of the code alphabet and assigns the codewords by traversing the tree. Practically speaking, we can summarize the algorithm as follow:

1. If not available, compute the symbols probabilities by scanning the text.
2. Sort the symbols according to their probabilities, non-increseingly.
3. Make sure that the number of source symbols n is $d + \alpha(d - 1)$, for some integer α . If needed add *dummy* symbols with probability 0.
4. Merge the least d probable symbols in to a node until a root is formed.
5. Assign the codeword by traversing the tree.

Is it obvious that encoding a text with Huffman has a significant overhead depending on the code alphabet. We must infact transmit, in addition to the bit stream, also the tree. Decoding the code is straightforward as well: we simply traverse the tree and decode each symbol.

In terms of complexity Huffman encoding/decoing takes $\mathcal{O}(n \log n)$, where n is the number of source symbols.

In addition to the tree overhead, we have to consider the possibility in which the tree gets corrupted. In the next few sections we consider two variats of the plain Huffman encoding that try to fix these issues.

¹At that time, Fano and Shannon were looking for a technique to build optimal codes.

Section 2 – Huffman encoding

Example: Consider the source $S = \{a, b, c, d, e, f, g\}$, where $\Pr(a) = 0.3, \Pr(b) = 0.125, \Pr(c) = 0.125, \Pr(d) = 0.05, \Pr(e) = 0.1, \Pr(f) = 0.1, \Pr(g) = 0.2$. Provide a ternary Huffman code for S .

To begin with, let us sort the source symbols; we have: $a \succeq g \succeq b \succeq c \succeq e \succeq f \succeq d$. Since the number of symbols is of the form $d + \alpha(d - 1) = 3 + 2 * 2$, we can move on with merging the nodes. Thus, we consider the three least probable symbols and we merge them; in our case the symbols d, e, f . Iterating the procedure we get the tree in Figure 2.

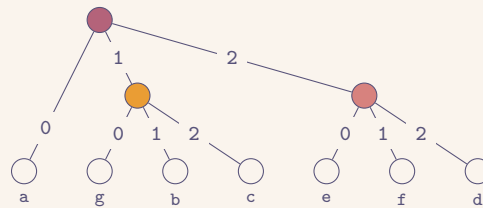


Figure 2: Huffman tree for the source S .

Therefore, we get the code $\mathcal{C} = \{0, 10, 11, 12, 20, 21, 22\}$.

Assume now receiving the stream 222122111101020; knowing that the tree used is the one above, what is the encoded string? By trivially traversing the tree, we can easily decode the stream as *dfdbbage*.

– 2.1 – Canonical Huffman encoding.

As stated previously, Huffman decoding needs the tree used in compression to correctly decode the bit-stream; it's obvious that, the larger the source alphabet is, the larger is the outputted tree. Can we avoid transmitting the whole tree then? In the present section we describe a variation of the classical Huffman encoding that does exactly so. We'll show that all that we need are the symbols to transmit, ordered non-increasingly, and the first codeword for each length.

Before we proceed, let us observe something: there exist optimal codes that correspond to no Huffman tree, for instance the tree on the right in Figure 3, corresponds to no Huffman encoding, yet it's optimal.

Let's now consider how to compute the Canonical Huffman encoding. Starting from the Huffman encoding of the source, compute the length of the codewords and construct the following data structures:

- A `num` array, to store at `num[l]` the number of codewords of length l .
- A `symb` array, that at `symb[l]` stores the list of symbols having codeword length of l -bits.

Section 2 – Huffman encoding

- A fc array, storing at $fc[l]$ the first codeword off all symbols encoded with l -bits.

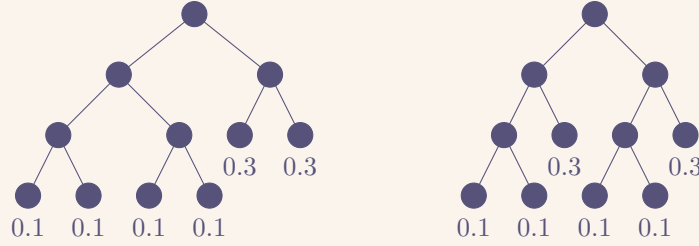


Figure 3: Comparison of Huffman tree and an equivalent optimal tree.

Using the above structures, we assign consecutive codewords to the symbols in $\text{symb}[l]$ starting from $fc[l]$. All the arrays are 1-based.

Building the num and symb array is trivial, the question is how to build fc . Let MAX be the maximal codeword length, then $fc[\text{MAX}] = 0$, as a consequence of the tree we are building, for any $l \leq \text{MAX}$, $fc[l] = (fc[l+1] + \text{num}[l+1]) / d$, where d is the size of the alphabet. The pseudo-code in Procedure `buildFC` uses a bottom-up approach, hence the tree built is unbalanced to the left.

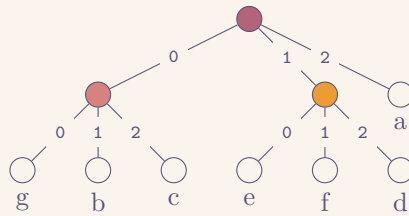
Procedure `buildFC`

```

 $fc[\text{MAX}] = 0$ 
for  $l \leftarrow \text{MAX} - 1$  to  $1$  do
     $fc[l] \leftarrow (fc[l+1] + \text{num}[l+1]) / d$ 
     $l \leftarrow l - 1$ 
end

```

Example: Assume we want to compute the Canonical Huffman encoding for the tree in Figure 2. It follows easily that $\text{num} = \{1, 6\}$ and $\text{symb} = \{a, \{g, b, c, e, f, d\}\}$. We now proceed building the fc array. By using the procedure we have $fc[2] = 0$ while $fc[1] = (fc[2] + \text{num}[2]) / 3 = 2$. The tree we obtain, symmetric to that of Figure 2, is shown below.



To conclude our discussion, we provide the Procedure `decodeCanonicalHuffman` to decode string encoded using Canonical Huffman. Recall that the

Section 2 – Huffman encoding

decoder has access to both `fc` and `symb`; therefore, we can simply read bits until the value is less than `fc[l]`, for all l .

Procedure decodeCanonicalHuffman

<pre><i>v</i> ← <i>next_bit</i>() <i>l</i> ← 1 while <i>v</i> < <i>fc</i>[<i>l</i>] do <i>v</i> ← 2 * <i>v</i> + <i>next_bit</i>() <i>l</i> ← <i>l</i> + 1 end return <i>symb</i>[<i>l</i>, <i>v</i> - <i>fc</i>[<i>l</i>]]</pre>
--

– 2.2 – Adaptive Huffman encoding.

In the previous section we discussed Canonical Huffman encoding, which solves the issue of transmitting the tree, but not that of scanning the text twice. In this section we shall present a variant of Huffman encoding that solves this issue.

Proposed independently by Fallar in 1973 and Gallager in 1978, later improved by Knut, the Adaptive Huffman encoding solves the issue of repeated scans of the text own of the classic approach. The core idea is simple: build a optimal Huffman tree for the text process thus far, and reorganize the tree when necessary.

More in details, we can summarize the procedure as follow:

1. Let T be an empty tree, or let it contain a special symbol NYT.
2. While scanning the text, for each symbol:
 - If it has already occurred, use the available codeword to encode it;
 - If not, encode the NYT symbol, then add the symbol as a new leaf and encode it.
3. After encoding a symbol, update T as follow:
 - Increment the frequency of the symbol and all its ancestors;
 - Reorganize the tree to maintain the Huffman properties.

It can be proved that, in general, the adaptive approach and the classic one have the same performances.

– 3 – Arithmetic encoding.

In Section 2, we said that Huffman encoding was proposed by D. A. Huffman as answer to Fano's question. Huffman's answer wasn't the only one. In fact another of Fano's student, Peter Elias, suggested a technique which came to be known as Arithmetic encoding.

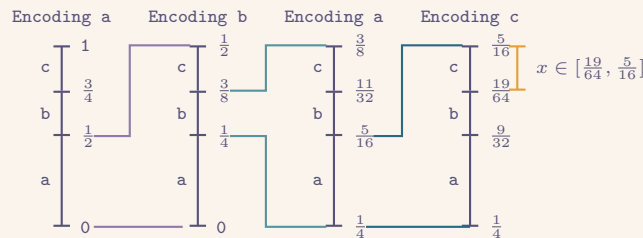
The core idea is extremely simple: instead of defining a codeword for each symbol, we encode the whole text in one go. More precisely, the text is encoded as decimal number in the range $[0, 1]$.

Algorithmically speaking, we proceed as follow: divide the range $[0, 1]$ in to n subintervals, each proportional to the probability of a distinct symbol, which we can assume to be lexicographically sorted; each time a symbol of the input text is read, select the corresponding subintervals and repeat the splitting procedure. Once the whole text is read, output a decimal number within the last interval.

Below we provide the pseudo-code of the procedure just described.

Procedure encode(S, n, Prob)
$l \leftarrow 0$ $s \leftarrow 1$ for $i \leftarrow 1$ to n do $s \leftarrow s * \text{Prob}[S[i]]$ $l \leftarrow l + s * f_{S[i]}$ /* Assume $f_{S[i]}$ to be the cumulative distribution function */ $i \leftarrow i + 1$ end return $x \in [l, l + s]$

Example: Consider the string $S = abac$, how do we encode it? By apply the encoding procedure we get the following.



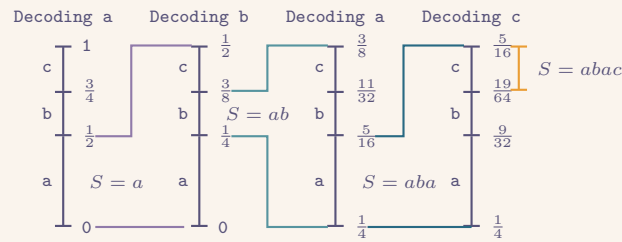
The question now is: how many bits do we need? Some result in number theory, allows us to say that we need exactly $\lceil \log^2/s \rceil$ bits.

Section 3 – Arithmetic encoding

The decoding process is symmetric: the range $[0, 1]$ is divided in n subintervals, at each iteration the proper range is selected and the intervals recomputed accordingly. The process is valid because the procedure is synchronized; that is, both encoder and decoder know the source alphabet and the associated probabilities. Below the pseudo-code of the procedure to decode a string encoded with Arithmetic encoding.

Procedure decode(Stream, n, Prob) $l \leftarrow 0$ $s \leftarrow 1$ $S \leftarrow ""$ for $i \leftarrow 1$ to n do $S \leftarrow S + \sigma$ /* Assume + to be the concatenation operation and σ the read symbol. */ $s \leftarrow s * Prob[S[i]]$ $l \leftarrow l + s * f_{S[i]}$ /* Assume $f_{S[i]}$ to be the probability cumulative function */ end return S

Example: Consider $x = 39/128$. If the source symbols and probabilities are that of the previous example, what is the decoded string?



– 3.1 – Adaptive arithmetic encoding.

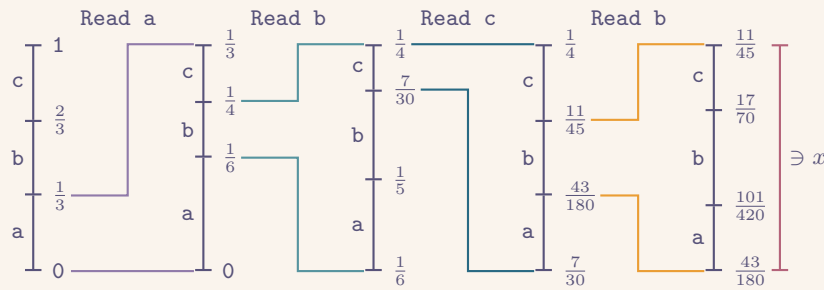
As for Huffman encoding, the arithmetic encoding procedure just discussed has an issue: the probabilities needs to be known before the encoding begins. A question then arises: can we do better as for the adaptive Huffman encoding? That is, can we make arithmetic encoding adaptive? In the present section we discuss how we can achieve adaptability.

The idea is the same as that of adaptive Huffman, but instead of updating the tree, we update the known probabilities. Algorithmically speaking, it

Section 3 – Arithmetic encoding

begins by assuming that all symbol have the same probability; that is, the n subintervals have the same width. As for the static case, once a symbol is read it selects the corresponding subinterval, and increases that symbol probability. The selected interval is then divided in n subintervals with width proportional to the probabilities.

Example: Let $S = abcb$ be the text to encode. Before any character is read, the probabilities are: $p_a = p_b = p_c = 1/3$. Once the first character (a) is read, we select the range $[0, 1/3]$ and update the probabilities as $p_a = 1/2, p_b = p_c = 1/4$. We repeat this process until the end of the text, encoding S within the range $[43/180, 11/45]$.



The decoding step is symmetrical: instead of updating the probabilities once a symbol is read, these are updated each time a new one is decoded. Apart from this caveat, both the encoding and decoding procedures are equal to Procedure encode and Procedure decode.

– 4 – Universal encoding of integers.

Many applications use integers to represent data. For instance, when we transmit an image, what we actually transmit are triplets of integers in the range $[0, 255]$. Additionally, many theoretical results we shall see in the following, make use of integers (see Section 5 and ??).

Therefore the questions: can we define a optimal code for integers? Can we use the codes seen so far? Both questions have an affirmative answer, but before analyzing it, we formalize the problem.

Let $S = s_1 s_2 \dots$ be a sequence of (positive) integers. Is there a prefix code such that the average codeword length is $\mathcal{O}(\log s_i)$, for any integer s_i .

In the subsequent sections we describe three of such code, while we use the remainder of the present section to make some remarks.

Remark: We can relax the constrain on positive integers. In fact, we can simply proceed to define a bijective functions that maps each integer to another. One of such map is the following:

$$f(n) = \begin{cases} 2n, & \text{if } n > 0 \\ -2n + 1, & \text{otherwise.} \end{cases}$$

Remark: Why don't we use the binary representation? It easily seen that this kind of representation is not prefix free. For instance, if we consider the binary representation of 2 and 4 we have, respectively, 10 and 100 which are prefix one of another.

A simple solution is the unary encoding. Essentially, any positive integer n is encoded as a sequence of $n - 1$ zeros followed by a one. That is

$$U(n) = \underbrace{00 \dots 0}_{n-1 \text{ times}} 1.$$

The code is trivially prefix-free, as any two different integers differ by at least one digit. At the same time its use isn't feasible, as the number of bits needed to encode the integer increases linearly.

Before we describe any particular family of codes to encode integers, let us consider Huffman encoding. Can we use it? It's known by now that Huffman encoding performs well when the probabilities are known a priori. Hence, if the distribution of integers is known, Huffman encoding is still worth considering. On the other hand, if the distribution is unknown, or it may vary, then Huffman works poorly and special codes should be considered.

– 4.1 – Elias codes.

The first family of universal codes for integers we consider are *Elias codes*. Proposed in [3], consists of three universal codes for integers, namely, the

Section 4 – Universal encoding of integers

gamma, the delta and the omega encoding.

In what follow we consider only the gamma and delta coding, and refer the reader to the paper for the omega encoding.

– 4.1.1 – Gamma encoding.

Let x be an integer and let $B(x)$ be it's binary expansion, i.e. it's binary representaion. Then, the gamma encoding of x consist of two parts: the unary representation of $|B(x)|$ and the binary representation of x with the most significant digit removed. That is

$$\gamma(x) = U(|B(x)|)\tilde{B}(x)$$

where $\tilde{B}(x)$ is $B(x)$ with the most significant digit removed.

Example: Consider the integer 29. Its binary representation $B(29) = 11101$. The unary encoding of $|B(29)|$ is $U(|B(29)|) = U(5) = 00001$. Therefore, the gamma encoding of 29 is

$$\gamma(29) = \underbrace{00001}_{U(|B(29)|)} \underbrace{1101}_{\tilde{B}(29)}$$

Decoding is trivial: count the number of zeros up to the first 1, say this are k , fetch the following $k + 1$ bits and interpret the sequence as the integer x .

It can be proved that gamma encoding is optimal when the distibution is

$$p(x) = \frac{1}{x^2}.$$

As easily seen, the inefficiency is given by the unary encoding of $|B(x)|$.

– 4.1.2 – Delta encoding.

Defined starting from gamma encoding, delta encoding allows to define shorter codewords². By using the same notation as before, if x is an integer its gamma encoding consists of two parts: the gamma encoding of $|B(x)|$ and $\tilde{B}(x)$. That is

$$\delta(x) = \gamma(|B(x)|)\tilde{B}(x).$$

Example: Consider again 29. Its binary representaion is $B(29) = 11101$. The gamma encoding of $|B(29)|$ is

$$\begin{aligned} \gamma(29) &= U(|B(|B(29)|)|)\tilde{B}(|B(29)|) \\ &= U(|B(5)|)\tilde{B}(5) \\ &= 00101. \end{aligned}$$

²It should be pointed out that this is true for values of $n \geq 32$.

Section 4 – Universal encoding of integers

We can conclude that the delta encoding of 29 is

$$\delta(29) = \underbrace{\overbrace{001}^{U(|B(5)|)} \overbrace{01}^{\tilde{B}(5)}}_{\gamma(|B(29)|)} \underbrace{1101}_{\tilde{B}(29)}$$

Decoding is simple: count the number of zeros up to the first 1, say these are k , fetch the following $k + 1$ and interpret the sequence as the integer x ; fetch the following $x - 1$ bits, prepend a 1 to the sequence and interpret it as the integer n .

It can be proved that delta encoding is optimal when the distribution is

$$p(x) = \frac{1}{x \log^2 x}.$$

Remark: What can we say about the sensitivity of gamma and delta encoding? That is, how well do the two encoding do when errors occur in transmission? It's easily seen that, even if a single gets flipped, the whole sequence gets decoded incorrectly.

The main issue is that neither of the two encoding produces self-synchronizing codewords. In the next section we describe a code that instead does produce self-synchronizing codewords

– 4.2 – Fibonacci codes.

Introduced in 1989 by R. Capocelli, Fibonacci encoding produces self-synchronizing codewords. The encoding is based on the Zeckendorf theorem stated below, which allows us to represent each positive integer as the sum of Fibonacci numbers. Recall that Fibonacci numbers are defined by the recurrence

$$F_n = \begin{cases} 0, & \text{if } n = 0 \\ 1, & \text{if } n = 1 \\ F_{n-1} + F_{n-2}, & \forall n \geq 2. \end{cases}$$

Theorem (Zeckendorf's): *Let n be an integer. Then, n is either a Fibonacci number or there exist distinct non-consecutive Fibonacci numbers $F_{i_1}, F_{i_2}, \dots$ such that*

$$n = F_{i_1} + F_{i_2} + \dots$$

with $F_{i_j} \geq 1$.

Zeckendorf's representation has the property that no two consecutive bits are set to 1.

Let n be an integer. To encode it using Fibonacci encoding: find the greatest Fibonacci number lower than or equal to n , say it's F_{i_j} , and compute the

Section 4 – Universal encoding of integers

remainder $r = n - F_{i_j}$. Starting from the left, set the i_j -th bit to 1. Keep computing the remainder replacing at each step r to n . Append a 1 to the sequence.

Appending a 1 to the representation, allows us to have self-synchronizing codes. If errors occur in transmission, the affected codeword is discarded without affecting the rest of the bitstream.

Example: Consider the integer 177. To begin with compute all Fibonacci numbers lower than or equal to 177, these are: $F_1 = 1, F_2 = 2, F_3 = 3, F_4 = 5, F_5 = 8, F_6 = 13, F_7 = 21, F_8 = 34, F_9 = 55, F_{10} = 89, F_{11} = 144$. Take F_{11} and compute $r = 177 - 144 = 33$, and set the 11-th bit from the left to 1. We have for the moment $Fib(177) = 00000000001$. By computing r once more we have $r = 33 - 21 = 12$; since the 7-th Fibonacci number was used we update the encoding by setting the 7-th bit to one. Repeating the process we get that $Fib(177) = 101010100011$.

Decoding is trivial: locate the “11” in the codeword, and remove the extra bit. Replace each 1 within the codeword with the corresponding Fibonacci number, and compute the integer x as the sum of these values.

Fibonacci encoding is proved optimal for

$$p(x) = \frac{1}{x^{1.44}}.$$

– 5 – Burrows-Wheeler transform.

Introduced in [2], Burrows-Wheeler Transform (BWT) is a technique that allows a better compression. The transformation is simple, as described in the following. Let w be the text we want to transmit; as first step we append a end of string symbol to it, let it be \$, we then compute all cyclic rotations of w , and sort all cyclic rotations lexicographically.

For the sake of simplicity, assume the cyclic rotations to be displayed in a matrix, see for instance Table 1. Then, output of the BWT is the last column of said matrix and the row index I of the original text.

Example: Let $w = \text{mississippi}$. Computing all w 's cyclic rotations we get:

```
$mississippi
i$mississipp
pi$mississip
ppi$mississi
ippi$mississ
sippi$missis
ssippi$missi
issippi$miss
sissippi$mis
ssissippi$mi
ississippi$m
mississippi$
```

Sorting them lexicographically, and displaying them in a matrix we get the following.

	F										L	
0	\$	m	i	s	s	i	s	s	i	p	p	i
1	i	\$	m	i	s	s	i	s	s	i	p	p
2	i	p	p	i	\$	m	i	s	s	i	s	s
3	i	s	s	i	p	p	i	\$	m	i	s	s
4	i	s	s	i	s	s	i	p	p	i	\$	m
5	m	i	s	s	i	s	s	i	p	p	i	\$
6	p	i	\$	m	i	s	s	i	s	s	i	p
7	p	p	i	\$	m	i	s	s	i	s	s	i
8	s	i	p	p	i	\$	m	i	s	s	i	s
9	s	i	s	s	i	p	p	i	\$	m	i	s
10	s	s	i	p	p	i	\$	m	i	s	s	i
11	s	s	i	s	s	i	p	p	i	\$	m	i

Table 1: Matrix representation of the cyclic rotations of “mississippi\$”.

Therefore, the output is $\text{bwt}(w) = \langle ipssm$piissii, 6 \rangle$.

The BWT satisfies some interesting properties, that are fundamental to compute the inverse transformation. These, listed below, allow us to compute two distinct mappings, namely, the FL and the LF mapping.

Section 5 – Burrows-Wheeler transform

1. For every $i \neq I$, $L[i]$ is followed by $F[i]$.
2. The first character of w is $F[I]$.
3. The i -th occurrence of character x in L corresponds to the i -th occurrence of x in F .

Assume $\langle L, I \rangle$ to be the output of the BWT of some string. From it, computing F is trivial – simply sort lexicographically the symbols in L . By property 3 we can define a mapping from F to L , obtaining this way a permutation, call it τ . Then, it's easily seen that the i -th symbol of w is given by $F[\tau^i[I]]$. The LF -mapping is obtained similarly, but the string obtained is in reverse.

Example: Consider $\langle h\$lpae, 1 \rangle$. By sorting L , we get: $F = [\$, a, e, h, l, p]$, from which

$$\tau = \begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 \\ 1 & 4 & 5 & 0 & 2 & 3 \end{pmatrix}.$$

Retrieving w , we have: $w[0] = F[\tau^0[I]] = F[I] = a$, $w[1] = F[\tau[I]] = F[4] = l$, $\dots$, $w[5] = F[\tau^5[I]] = F[0] = \$$. In conclusion, $w = aleph\$$ from which we remove the end of string symbol $\$$.

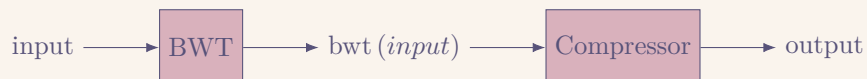
Remark: It is easily seen that BWT's efficiency strictly depends on the sorting of the cyclic rotations. It is well known that, by using a standard sorting algorithms the best we can achieve is a $\mathcal{O}(n \log n)$ time complexity, which is obviously too much for a pre-processing step.

It can be proved that if we focus on sorting the suffixes of the string, we have effectively sorted all cyclic rotations. How should we proceed then? In many applications we achieve this by computing the Suffix Array (SA); that is, a sorted array of the positions of the suffixes. In Section 5.2 we show how to efficiently compute the SA, while in the next section we focus on analyzing some theoretical aspects of BWT.

– 5.1 – Analysis of the BWT.

Why is BWT so useful? The main perk of BWT is that its output is locally homogeneous; that is, it is the concatenation of several substrings with few runs of equal symbols.

Consider the structure of a BWT-based compressor shown below, where



Compressor refers to a statistical compressor, such as Huffman or Arithmetic

Section 5 – Burrows-Wheeler transform

coding using Move-To-Front (MTF) and Run-Length-Encoding (RLE) as pre-processor. Both are briefly described in the following section.

In their paper, Ferragina and Manzini suggested that MTF was a needed step, but in [6] Giancarlo and Sciortino proved otherwise.

A detailed analysis of BWT-based compressor is provided by Manzini in [10]; in what follows we summarize some of his results.

Manzini analyzes the compression ratio of BWT-based compressor in terms of 0-order empirical entropy, defined as the quantity

$$H_0(S) = - \sum_{i=0}^k \frac{n_i}{n} \log \frac{n_i}{n}$$

where k is the number of distinct symbols in S , and n_i the number of occurrences of the i -th symbol. By using this definition, which makes no assumption on the probability distribution of the source, empirical entropy can be used for a worst case analysis. Using empirical entropy Manzini proved that

$$\begin{aligned} |Huff(S)| &\leq |S|H_0(S) + |S| \\ |Arit(S)| &\leq |S|H_0(S) + \mu|S| \end{aligned}$$

where $|S|H_0(S)$ is the size of an optimal compressor that uses $n_1/n \log \frac{n}{n_i}$ bits per symbol. The Arithmetic bound holds if finite-precision is used.

The empirical entropy can be generalized to any order, and is defined as

$$H_k(S) = \frac{1}{|S|} \sum_{w \in \Sigma^k} |w_s| H_0(w_s)$$

where w_s is the concatenation of the symbols immediately following w in S . For instance, if the string is “mississippi” and $k = 3$, we have $\mathbf{mis}_S = s$, $\mathbf{iss}_S = ii$, $\mathbf{ssi}_S = sp$, $\mathbf{siss}_S = s$, $\mathbf{sip}_S = p$, $\mathbf{ipp}_S = i$ and $\mathbf{ppi}_S = \$$. Computing the 3-order empirical entropy we have

$$\begin{aligned} H_3(\text{mississippi}) &= \frac{1}{11} [H_0(\mathbf{mis}_S) + 2H_0(\mathbf{iss}_S) + 2H_0(\mathbf{ssi}_S) + H_0(\mathbf{siss}_S) \\ &\quad + H_0(\mathbf{sip}_S) + H_0(\mathbf{ipp}_S) + H_0(\mathbf{ppi}_S)] \\ &= \frac{2}{11} \approx 0.18. \end{aligned}$$

Essentially, BWT allows us to boost a 0-order compressor, such as Huffman, making it behave like a compressor which uses a memory of length k .

Some additional results proved in Manzini’s paper are the following:

- Let $bwt_0(S)$ be the output of the compressor $Order_0(mtf(bw(S)))$, then it holds

$$|bwt_0(S)| \leq 8|s|H_k(s) + (\mu + 2/25)|S| + h^k(2h \log h + 9)$$

Section 5 – Burrows-Wheeler transform

where h is the size of the alphabet.

- Let $btw_{RL}(S)$ be the output of the compressor $Order_0(rle(mtf(bw(S))))$, then it holds

$$|bwt(w)_{RL}(S)| \leq (5 + 3\mu)|S|H_k^*(S) + g_k$$

where g_k is a constant dependent on k and h , and $H_k^*(S)$ is the k -th order modified empirical entropy; that is, the empirical entropy whose factor are of length up to k .

– 5.1.1 – MTF and RLE.

Our description of move-to-front and run-length-encoding will be the following: we describe MTF in details, presenting the core idea and an example; for RLE we simply give the general idea.

Introduced in [1], MTF is a simple encoding based on frequency. Formally, let S be a text and let A be the alphabet on which its defined. It follows, that S 's characters satisfy some lexicographic order. Using this fact, MTF encodes the i -th symbol in S by the number of distinct symbols preceeding it in A , then moving said character to the front of A . The process is repeated until S is scanned completely.

The core idea is that, if S is locally homogeneous, as in the case of BWT, then the output of the encoding will be a sequence of small integers, which can be encoded in turns by some universal encoder of integers.

Decoding is easy as well; since the alphabet is known by both parties, we proceed as done in encoding. Precisely, we decode the symbol as the $(i + 1)$ -th symbol in A and move that character to the front.

Perfomance-wise it can be proved that MTF behaves slightly worse then Huffman in the worst case, and a lot better in the best one.

Example: Consider the string $w = smmihtt$ecaa = bwt(mathematics\$)$. The alphabet is $A = \{\$, a, c, e, h, i, m, s, t\}$ and the lexicographic order is the usual one ($\$ \prec a \prec \dots \prec t$).

We encode s as 7, and move it to the fron of A ; m is then encoded as 7, and so on. Once w has be read completely, the output of MTF is the sequence $(7, 7, 0, 7, 7, 8, 0, 5, 8, 8, 0)$. Note that w has little-to-no run, thus the “big” integers are expected.

Run-length-encoding is somewhat simple, and several implementations can be found in the literature. The idea is that, if a character x appears n consecutive times, then it can be encoded as nx . How to encode nx depends on the implementations, some use pairs eg. (x, n) ; other encode n replacing the 1s in its binary representation with x .

– 5.2 – Efficient computation of the SA.

Before we describe the algorithm let us make a remark: suffix arrays are not the only option available when it comes to sorting suffixes. One could also use a Suffix Tree (ST), whose construction has been proposed in several papers; see [4, 5, 11, 13] for instance. The only issue is that, ST are space inefficient.

The algorithm we describe here was proposed by Kärkkäinen et al. in [9]. Broadly speaking, the problem of sorting suffixes of a string of length n , is recursively reduced to that of sorting the suffixes of a string of length $\frac{2}{3}n$.

The algorithm makes the following trivial assumption: the string is defined over a integer alphabet. Using the above assumption, we use $[i, j]$ as a shorthand for $\{1, \dots, j\}$.

Let $T[0, n]$ be the text to process. Append a end of string symbol (\$) to it. For $k = 0, 1, 2$, define

$$B_k = \{i \in [0, n+1] \mid i \equiv k \pmod{3}\}.$$

Let R_k denote the string $T[k, n]$, for $k = 1, 2$. Consider R_k 's characters as triplets of the form $[t_i, t_{i+1}, t_{i+2}]$, eventually padded by an end of string symbol, for $i \in B_k$. Let $R = R_1 R_2$. It can be shown that there exist a correspondence between the suffixes of R and the suffixes of S .

To sort R 's suffixes, radix sort its characters (the triplets) and replace each with its rank; denote this new string with R' . If all characters are distinct, the ranks give the order of the suffixes; otherwise sort R' suffixes using the DC3 algorithm.

After the suffixes have been computed, for $i \in B_1 \cup B_2$, let $\text{rank}(S_i)$ denote the rank of suffix S_i . For $i \in B_0$, $\text{rank}(S_i)$ is undefined.

Note that for $i \in B_0$, S_{i+1} has been computed. Thus, if we represent each S_i as the pair $(t_i, \text{rank}(S_{i+1}))$, it follows that for all $i, j \in B_0$,

$$S_i \leq S_j \iff (t_i, \text{rank}(S_{i+1})) \leq (t_j, \text{rank}(S_{j+1})).$$

At this point all suffixes whose starting position belongs to $B_1 \cup B_2$ have been sorted, and we have a way to sort suffixes starting at a position in B_0 . What we are left with is merging the two. For the sake of simplicity, let $j \in B_0$ and $i \in B_1 \cup B_2$. Then, to compare S_i and S_j we do the following:

- if $j \in B_1$, compare S_i and S_j as both $i, j \in B_0$. That is

$$S_i \leq S_j \iff (t_i, \text{rank}(S_{i+1})) \leq (t_j, \text{rank}(S_{j+1})).$$

- if $j \in B_2$, compare t_i and t_j , if these are equal compare t_{i+1} and t_{j+1} and if these are equal compare $\text{rank}(S_{i+2})$ and $\text{rank}(S_{j+2})$. That is,

$$S_i \leq S_j \iff (t_i, t_{i+1}, \text{rank}(S_{i+2})) \leq (t_j, t_{j+1}, \text{rank}(S_{j+2})).$$

Section 5 – Burrows-Wheeler transform

Computationally speaking, apart from the recursive call, all steps take linear time. The recursive call is on a string of length $\frac{2}{3}n$. Therefore the overall time complexity is given by the recurrence $T(n) = T(\frac{2}{3}n) + \mathcal{O}(n)$, whose solution by the Master theorem is $\mathcal{O}(n)$ time.

Example: Consider the string $w = \text{mathisawesome\$}$. We get the buckets:

$$B_0 = \{0, 3, 6, 9, 12\}, B_1 = \{1, 4, 7, 10, 13\}, B_2 = \{2, 5, 8, 11\}$$

For $k = 1, 2$, we get

$$\begin{aligned} R_1 &= [a, t, h][i, s, a][w, e, s][o, m, e][\$, \$, \$] \\ R_2 &= [t, h, i][s, a, w][e, s, o][m, e, \$] \end{aligned}$$

thus,

$$\begin{aligned} R &= R_1 R_2 \\ &= [a, t, h][i, s, a][w, e, s][o, m, e][\$, \$, \$][t, h, i][s, a, w][e, s, o][m, e, \$]. \end{aligned}$$

Using the radix sort on R 's characters we get

$$R' = (1, 3, 8, 5, 0, 7, 6, 2, 4).$$

since no two are equal, the suffixes are sorted. What we are left is computing the suffixes for the bucket B_0 . By using the notation above, these are

$$\begin{aligned} S_0 &= (m, \text{rank}(S_1)) = (m, 1) \\ S_3 &= (h, \text{rank}(S_4)) = (h, 3) \\ S_6 &= (a, \text{rank}(S_7)) = (a, 8) \\ S_9 &= (s, \text{rank}(S_{10})) = (s, 5) \\ S_{12} &= (e, \text{rank}(S_{13})) = (e, 0). \end{aligned}$$

Lastly, comparing them with the previously computed suffixes, the suffix array for w is

$$SA_w = (13, 1, 6, 12, 8, 3, 4, 0, 11, 10, 5, 9, 2, 7)$$

since $[\$, \$, \$]$ represents the end of string symbol, we can remove S_{13} .

ToDo: complete.

References.

- [1] Jon Louis Bentley et al. “A Locally Adaptive Data Compression Scheme”. In: *Commun. ACM* 29.4 (1986), pp. 320–330. DOI: [10.1145/5684.5688](https://doi.org/10.1145/5684.5688).
- [2] M. Burrows and D. J. Wheeler. *A Block-sorting Lossless Data Compression Algorithm*. Tech. rep. Research Report 124. Palo Alto, California: Digital Equipment Corporation, Systems Research Center, May 1994. DOI: [10.1109/DCC.1997.582009](https://doi.org/10.1109/DCC.1997.582009).
- [3] Peter Elias. “Universal Codeword Sets and Representations of the Integers”. In: *IEEE Transactions on Information Theory* 21.2 (1975), pp. 194–203. DOI: [10.1109/tit.1975.1055349](https://doi.org/10.1109/tit.1975.1055349).
- [4] Martin Farach. “Optimal Suffix Tree Construction with Large Alphabets”. In: 1997, pp. 137–143. DOI: [10.1109/SFCS.1997.646102](https://doi.org/10.1109/SFCS.1997.646102).
- [5] Martin Farach and S. Muthukrishnan. “Optimal Logarithmic Time Randomized Suffix Tree Construction”. In: 1996, pp. 550–561. DOI: [10.1007/3-540-61440-0_158](https://doi.org/10.1007/3-540-61440-0_158).
- [6] Raffaele Giancarlo and Marinella Sciortino. “Optimal Partitions of Strings: A New Class of Burrows-Wheeler Compression Algorithms”. In: 2003, pp. 129–143. DOI: [10.1007/3-540-44888-8_10](https://doi.org/10.1007/3-540-44888-8_10).
- [7] R. V. L. Hartley. “Transmission of Information”. In: *Bell System Technical Journal* 7.3 (1928), pp. 535–563. DOI: [10.1002/j.1538-7305.1928.tb01236.x](https://doi.org/10.1002/j.1538-7305.1928.tb01236.x).
- [8] David A. Huffman. “A Method for the Construction of Minimum-Redundancy Codes”. In: *Proceedings of the IRE* 40.9 (1952), pp. 1098–1101. DOI: [10.1109/JRPROC.1952.273898](https://doi.org/10.1109/JRPROC.1952.273898).
- [9] Juha Kärkkäinen, Peter Sanders, and Stefan Burkhardt. “Linear work suffix array construction”. In: *J. ACM* 53.6 (2006), pp. 918–936. DOI: [10.1145/1217856.1217858](https://doi.org/10.1145/1217856.1217858).
- [10] Giovanni Manzini. “An analysis of the Burrows-Wheeler transform”. In: *Journal of the ACM* 48.3 (2001), pp. 407–430. DOI: [10.1145/382780.382782](https://doi.org/10.1145/382780.382782).
- [11] Edward M. McCreight. “A Space-Economical Suffix Tree Construction Algorithm”. In: *J. ACM* 23.2 (1976), pp. 262–272. DOI: [10.1145/321941.321946](https://doi.org/10.1145/321941.321946).
- [12] Claude E. Shannon. “A mathematical theory of communication”. In: *Bell Syst. Tech. J.* 27.4 (1948), pp. 623–656. DOI: [10.1002/J.1538-7305.1948.TB00917.X](https://doi.org/10.1002/J.1538-7305.1948.TB00917.X).

- [13] Esko Ukkonen. “On-Line Construction of Suffix Trees”. In: *Algorithmica* 14.3 (1995), pp. 249–260. DOI: [10.1007/BF01206331](https://doi.org/10.1007/BF01206331).

Acronyms.

ACL Average Codeword Length. 5

BWT Burrows-Wheeler Transform. 18

MTF Move-To-Front. 20

RLE Run-Length-Encoding. 20

SA Suffix Array. 19

ST Suffix Tree. 22

UD Uniquely Decodable. 3